AutoVenda — Venda de carros online com cadastro de veículos

Sistema web para uma loja de veículos seminovos. Tem duas partes:

- **Catálogo público**: busca, filtros e envio de proposta de compra, sem precisar de cadastro.
- **Painel da loja**: cadastro de veículos com foto, marcas, análise de propostas, reserva e registro de vendas.

Projeto Integrador Transdisciplinar em Engenharia de Software II.

Funcionalidades

Visitante	Vendedor	Administrador
Buscar e filtrar veículos (marca, preço, ano, combustível, câmbio)	Login	Tudo do vendedor
Ordenar e paginar	Cadastrar/editar veículos com foto	Excluir veículos
Ver detalhes	Aceitar/recusar propostas (reserva automática)	Cadastrar/excluir marcas
Enviar proposta	Registrar vendas e ver faturamento	

O status do veículo segue o ciclo **Disponível → Reservado → Vendido**. Veículos vendidos saem do catálogo automaticamente.

Tecnologias

- **Back-end:** Python 3.11, Flask 3, SQLAlchemy 2, no padrão **MVC**
- **Front-end:** HTML5 (Jinja2), CSS3 e JavaScript puros, layout responsivo
- **Banco:** SQLite (desenvolvimento) / PostgreSQL (produção)
- **Testes:** pytest + pytest-cov (back-end), `node:test` (front-end)

Instalação

Pré-requisitos

- Python 3.11 ou superior
- Node.js 18 ou superior, apenas para executar os testes do front-end
- Git, para clonar o repositório

1. Clonar e acessar o projeto

```
git clone https://github.com/Gerson201/autovenda.git
cd autovenda
```

2. Criar o ambiente virtual e instalar as dependências

```
python -m venv .venv
```

Ative o ambiente virtual:

```
# Windows PowerShell
.\.venv\Scripts\Activate.ps1
```

```
# Linux/macOS
source .venv/bin/activate
```

Em seguida, instale as dependências do Python:

```
pip install -r requirements.txt
```

3. Criar os dados de demonstração e iniciar a aplicação

```
flask --app run seed
flask --app run run
```

Abra `http://127.0.0.1:5000` no navegador. O comando `seed` cria marcas, veículos com imagens e os usuários de demonstração; ele pode ser executado novamente sem duplicar os registros existentes.

Perfil	E-mail	Senha
Administrador	admin@autovenda.com	Admin@123
Vendedor	vendedor@autovenda.com	Vende@123

Testes

```
pytest --cov=app                          # 158 testes: unitários, integração, sistema e regressão
node --test "tests/frontend/*.test.js"    # 13 testes do front-end
```

Suíte	Testes	O que cobre
`tests/test_unitario_util.py`	32	Conversões, CPF, formatação
`tests/test_unitario_models.py`	56	Regras de negócio isoladas
`tests/test_controllers.py`	39	Controller + Model + banco + template, via HTTP
`tests/test_integracao.py`	3	Fluxos completos e comando `seed`
`tests/test_revisao_tecnica.py`	28	Regressão dos defeitos achados na revisão de código
`tests/frontend/validacao.test.js`	13	Regras de validação do navegador

Para executar somente os testes do front-end pelo atalho do projeto:

```
npm test
```

Organização do projeto

O projeto segue o padrão MVC. As regras de negócio e as entidades ficam separadas das rotas e das telas, o que permite testar cada parte sem depender da interface.

```
├── app/
│   ├── __init__.py       # criação e configuração da aplicação Flask
│   ├── config.py         # configurações de banco, sessão, upload e ambiente
│   ├── models/           # MODEL: entidades e regras de negócio
│   ├── controllers/      # CONTROLLER: rotas e fluxo das requisições
│   ├── views/            # VIEW: templates, CSS, JS, filtros de formatação
│   ├── servicos/         # upload de imagens e freio de tentativas de login
│   ├── seed_imagens/     # fotos de demonstração e créditos das imagens
│   └── seguranca.py      # CSRF, login, perfis
├── database/             # schema.sql (PostgreSQL) e consultas
├── entrega/              # documentação das três situações do PIT, em PDF
├── tests/                # testes unitários, integração, controladores, regressão e JS
├── requirements.txt      # dependências Python
├── package.json          # comando de teste do front-end
└── run.py                # ponto de entrada da aplicação
```

A pasta `entrega/` reúne, em PDF, o planejamento e a modelagem (Situação 1), a documentação de desenvolvimento (Situação 2) e a evidência dos testes e validações (Situação 3).

Documentação

A documentação entregue está em PDF, na pasta `entrega/`.

Arquivo	Conteúdo
`00-AutoVenda-Visao-Geral-e-Instalacao.pdf`	Esta página em PDF: visão geral, instalação e organização do projeto
`01-Situacao-1-Planejamento.pdf`	Escopo e requisitos · Modelagem UML · IHC e protótipo · Banco de dados e dicionário de dados
`02-Situacao-2-Desenvolvimento.pdf`	Arquitetura MVC · Manual de uso
`03-Situacao-3-Testes-e-Validacao.pdf`	Plano de testes · Relatório de testes · Formulário dos testadores · Registro de correções · Revisão técnica de código
`04-Creditos-das-Imagens.pdf`	Autoria e licença das fotos de demonstração
`05-Evidencias-Testes-Automatizados.pdf`	Saída completa do `pytest` e do `node --test`

Publicar em produção

Antes de expor a aplicação na internet, defina duas variáveis de ambiente:

```
export AUTOVENDA_AMBIENTE=producao
export SECRET_KEY="$(python -c 'import secrets; print(secrets.token_hex(32))')"
export DATABASE_URL="postgresql+psycopg://usuario:senha@servidor/autovenda"
```

Com `AUTOVENDA_AMBIENTE=producao` a aplicação **recusa iniciar** se a `SECRET_KEY` for a de desenvolvimento, e passa a marcar o cookie de sessão como `Secure` (só trafega por HTTPS). O script `database/schema.sql` cria o esquema no PostgreSQL com as restrições, índices e a trigger de `atualizado_em`.

Créditos das imagens

As fotos dos veículos de demonstração vêm do Wikimedia Commons, sob licenças Creative Commons. Autores e licenças estão em `entrega/04-Creditos-das-Imagens.pdf`.

Vídeos

- Demonstração do sistema: **pendente de gravação pelo autor**.
- Mudanças após os testes com usuários: **pendente da rodada de validação com 5 testadores**.

Aviso

A documentação do código, parte dos testes e a revisão técnica de código documentada na Situação 3 foram realizadas com a utilização de IA. Todas as informações foram revisadas e aprovadas por um responsável humano.